

RESUMEN IA PRIMER PARCIAL

INTRODUCCIÓN, INTELIGENCIA ARTIFICIAL Y AGENTES INTELIGENTES – UNIDAD 1

Inteligencia: Es el conjunto de capacidades cognitivas que permite resolver situaciones, identificar problemas y procesar la información que nos rodea incluyendo el uso de experiencias anteriores para generar nuevo conocimiento.

Inteligencia Artificial: Capacidad de hacer tareas que, por el momento, los humanos hacen mejor.

Los autores actuales, en la práctica, reconocen la construcción de la IA en 4 cuadrantes de acuerdo a **parámetros** totalmente arbitrarios que giran en torno a dos dimensiones principales:

- **RAZONAMIENTO-CONDUCTA:** Esta dimensión se refiere a si el foco está en el proceso interno (como piensa o razona el sistema) o en el resultado externo (cómo actúa o se comporta el sistema).
- **EFICIENCIA HUMANO-IDEAL:** Esta dimensión compara el sistema con el desempeño humano.
 - o **Humano:** Buscar que el sistema imite el pensamiento o la conducta humana
 - o **Ideal (Racional):** Busca que el sistema actúe de la mejor manera posible, de forma lógica y coherente, sin necesariamente imitar a los humanos.

	EFICIENCIA HUMANO-IDEAL	
CONDUCTA - RAZONAMIENTO	Sistemas que piensan como humanos	Sistemas que piensan racionalmente
	Sistemas que actúan como humanos	Sistemas que actúan racionalmente

Pensar como humanos: En pensar como humanos tenemos cosas como el reconocimiento de imágenes de patrones y de situaciones humanas que para nosotros son simples en la y que en teoría aun no hemos podido replicar a la perfección.

Queremos no solo conseguir los resultados de manera lógica sino también la sinapsis que hacen nuestras neuronas ante dicho estímulo. Esto es mucho más abarcativo y tiene que ver con la **ciencia cognitiva**.

Actuar como humanos: En actuación humana es lo mismo, nada más que llevado a un nivel físico, no solo es cognitivo. Aquí es donde surgen pruebas como la de Alan Turing. Estas pruebas tratan a cerca de un operador que se comunica a través de una terminal con una IA y se espera que no pueda distinguir si

es una máquina o un humano con quien se está comunicando, definiendo a la máquina como un ser inteligente.

Pensar racionalmente: En pensar racionalmente nos encontramos en el campo de la lógica, de los silogismos y de Aristóteles.

“Sócrates es un hombre, todos los hombres son mortales, por lo tanto, Sócrates es mortal”

Bajo la premisa de que cualquier problema factible con la totalidad de la información tiene la capacidad de ser resuelto por un ser inteligente, los logistas se topan con obstáculos como que no siempre conocemos la totalidad de las premisas o que algunas pueden ser erróneas. A su vez se topan con que es necesario utilizar una notación lógica para expresar el problema y que dicho problema a veces es factible, pero requiere infinitos recursos para ser resuelto.

Estos algoritmos tienden a basarse en la simulación y en cómo ante la entrada de información conseguimos una respuesta más determinista ya que suele seguir la proposición lógica de:

Premisa – Conclusión

Actuar racionalmente: Actuar racionalmente abarca los Agentes Racionales. Son aquellos que pueden percibir su entorno, modificarlo y permanecer en el tiempo de manera prolongada con controles autónomos.

Los distinguimos de los programas habituales como entidades más allá de lo abstracto que realizan inferencias acerca de lo que los rodea.

¿PORQUÉ ES INTERESANTE LA IA?

Sentido de la Identidad: La IA nos ayuda a reflexionar sobre nosotros mismos. El hecho de que los humanos nos hayamos llamado "homo sapiens" (hombre sabio) resalta la importancia de nuestras habilidades mentales. Estudiar la inteligencia artificial nos permite comprender mejor la inteligencia natural, es decir, la nuestra.

Comprensión y Construcción de Entidades Inteligentes: La IA se dedica a comprender cómo funciona la inteligencia. A diferencia de la filosofía y la psicología (que también estudian la inteligencia), la IA no solo busca entenderla, sino también construirla.

Las entidades inteligentes que se construyen son valiosas por sí mismas. A pesar de ser un campo joven, la IA ya ha producido aplicaciones "importantes y sorprendentes".

Es un Problema Complejo, pero la Solución Existe: Se reconoce que la IA aborda uno de los problemas más complejos. A diferencia de otros desafíos científicos (como viajar más rápido que la luz), el investigador en IA tiene una "prueba contundente de que tal búsqueda es factible": el propio ser humano. Mirarse en un espejo es ver un ejemplo de sistema inteligente.

FUNDAMENTOS DE LA IA

Aunque la IA es un campo relativamente nuevo, se nutre de ideas, perspectivas y técnicas de disciplinas mucho más antiguas. Las principales son:

- **Filosofía:** Contribuye con las bases del razonamiento, la lógica y la naturaleza de la mente.
- **Matemáticas:** Proporciona las herramientas para la lógica, la computación, la probabilidad y los algoritmos.
- **Psicología:** Aporta la comprensión de cómo los seres humanos perciben, aprenden, razonan y actúan.
- **Neurología:** Ofrece el estudio del cerebro y el sistema nervioso, inspirando modelos como las redes neuronales.
- **Computación:** Es la base tecnológica que permite implementar los modelos y algoritmos de IA.
- **Lingüística:** Fundamental para el procesamiento del lenguaje natural y la comunicación entre humanos y máquinas.

BREVE HISTORIA Y ALGUNOS HITOS

1936: Máquina de Turing: Alan Turing crea un concepto teórico fundamental para las computadoras modernas.

1943: Primera Neurona Artificial: Warren McCulloch y Walter Pitts desarrollan un modelo matemático de neurona, un precursor de las redes neuronales.

1950: Test de Turing: Alan Turing propone una prueba para determinar si una máquina puede exhibir un comportamiento inteligente indistinguible de un ser humano.

1956: Dartmouth Workshop: John McCarthy organiza una reunión crucial donde se acuña el término "Inteligencia Artificial". Asisten figuras importantes como Marvin Minsky, Nathaniel Rochester y Claude Shannon. Este evento es considerado el nacimiento oficial del campo de la IA.

Optimismo Exagerado Inicial: La primera década de la IA estuvo marcada por un gran optimismo, con rápidos avances en problemas específicos como la demostración de teoremas y juegos simples.

Etapas de Depresión (AI Winter): Cuando se intentaron resolver problemas más complejos, como el sentido común o el lenguaje natural, los resultados no fueron los esperados. Esto llevó a un "desánimo generalizado y a la pérdida de inversiones", una fase conocida como "invierno de la IA".

Resurgimiento (años 80s) y Crecimiento Actual: La IA resurgió en los años 80 con los **Sistemas Expertos**, desde entonces, el campo no ha parado de crecer.

El avance actual se debe a una combinación de factores: la enorme cantidad de datos disponibles, el aumento de los recursos computacionales (procesamiento y almacenamiento) y los progresos en técnicas como las redes neuronales profundas (deep learning). Estos elementos han creado un "círculo virtuoso" de inversión y resultados visibles.

CORRIENTES DE LA IA

Desde sus inicios, la IA se dividió en dos enfoques principales que incluso se distanciaron, lo que fue perjudicial para el avance del campo.

Corriente Conexionista – Subsimbolica o ascendente

Liderada por investigadores de la Universidad Carnegie-Mellon.

Propone desarrollar modelos de comportamiento humano que **simulen el cerebro lo más posible**, con un enfoque cognitivo.

Sigue un diseño "ascendente": comienza desde los niveles más bajos (como el procesamiento de señales, imitando el funcionamiento neuronal) y va construyendo niveles superiores de inteligencia. La idea es replicar los pasos evolutivos de la inteligencia en los seres vivos. De esta corriente surgieron los modelos conexionistas y las redes neuronales inspiradas en la biología.

Corriente Tradicional – Simbólica o descendente

Liderada por Marvin Minsky y John McCarthy del MIT.

Busca obtener comportamientos "inteligentes" sin preocuparse por la semejanza con la biología.

Fue la corriente dominante en los primeros 20 años de la IA.

Sigue un diseño "descendente" o "de procesamiento de símbolos". Se basa en **representar el conocimiento mediante sentencias declarativas** (hechos y reglas) y la **deducción de consecuencias a partir de reglas de inferencia** (lógica).

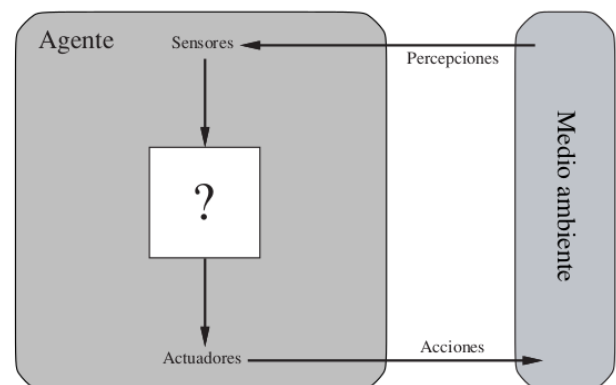
El problema de esta corriente fue que sus soluciones eran adecuadas para modelos pequeños, pero se volvían ineficientes o inaplicables **cuando se intentaban resolver problemas más grandes y complejos**, ya que la programación se volvía "crecientemente larga y compleja".

AGENTES INTELIGENTES o RACIONALES – UNIDAD 1

Este es un concepto fundamental en IA para modelar sistemas.

Un agente es "cualquier cosa capaz de percibir su medioambiente con la ayuda de sensores y actuar en ese medio utilizando actuadores".

- **Sensores:** Mecanismos a través de los cuales el agente recibe información del entorno (por ejemplo, una cámara, un micrófono, un teclado).
- **Actuadores:** Mecanismos a través de los cuales el agente realiza acciones en el entorno (por ejemplo, un motor, una pantalla, un altavoz).
- **Medio Ambiente:** El mundo o contexto en el que el agente opera.



Hipótesis de Trabajo: Se asume que un agente puede percibir sus propias acciones, aunque no siempre los efectos inmediatos de estas.

Noción de Agente: Es una "herramienta para el análisis de sistemas", lo que significa que es un modelo conceptual útil para diseñar y entender sistemas, no una forma estricta de clasificar todo lo que existe en el mundo.

Se asume que al ser agentes racionales sus acciones son correctas o que obtendrán el mejor desempeño posible. Esto debe ser evaluado para determinar la veracidad de este hecho, la **racionalidad** del mismo está determinada por 4 factores:

1. El éxito que tuvo al completar una operación
2. El conocimiento que está acumulado en el agente
3. Las acciones que toma para completar la acción
4. La secuencia de percepciones del agente hasta el momento

El símbolo de pregunta en el medio del diagrama corresponde con la función del agente que es el programa o caja negra que ante una secuencia de percepciones en un instante determinado se consigue algún efecto.

Es de destacar nuevamente, la autonomía que deben presentar estos agentes en general debe ser capaz de completar el conocimiento parcial o inicial que posee. Esto quiere decir que en cierto punto la información adquirida por la misma inteligencia se convertirá independiente de los datos con los que comenzó.

Un agente racional es aquel que hace lo correcto

Como primera aproximación, se puede decir que lo correcto es aquello que permite al agente obtener un mejor resultado

Se necesita determinar una forma de medir el éxito

AGENTE RACIONAL

"En cada posible secuencia de percepciones, un agente racional debe ejecutar aquella acción que supuestamente maximice su medida de rendimiento, basándose en las evidencias aportadas por la secuencia de percepciones y en el conocimiento incorporado que tenga el agente"

Un agente racional es aquel que "hace lo correcto". Lo "correcto" se interpreta como aquello que permite al agente "obtener un mejor resultado". Para ello, es fundamental tener una forma de "medir el éxito".

Especificación de la Tarea: Para definir un agente racional, se necesita especificar:

- **Descripción del entorno:** Cómo es el mundo en el que opera el agente.
- **Descripción de sensores y actuadores:** Qué puede percibir y qué puede hacer.
- **Medida de rendimiento:** Cómo se evalúa el éxito del agente.

Racionalidad (Dependencia de Factores): La decisión "racional" en un momento dado no es absoluta, sino que depende de cuatro elementos:

- **Medida de rendimiento:** El criterio de éxito establecido.
- **Conocimiento previo:** La información que el agente ya tiene sobre el entorno.
- **Acciones posibles:** Las opciones que el agente tiene para actuar.
- **Secuencia de percepciones:** El historial de lo que el agente ha percibido hasta ese momento.

A la hora de definir un agente es necesario entender, no solo las partes del autómata, sino aquello que está externo al mismo que son del entorno y del rendimiento que tiene con respecto al problema. Esta conjunción de palabras forma lo que se denomina como REAS (Rendimiento, Entorno, Actuadores y Sensores) siendo una forma de empezar a desarrollar los primeros modelos para agentes.

PROPIEDADES DE LOS ENTORNOS DE TRABAJO

Los entornos donde opera un agente pueden tener diferentes características, lo que influye en cómo se diseña el agente.

Totalmente observable vs. Parcialmente observable

- **Totalmente observable:** Hay sistemas que tienen la capacidad de saber todo acerca del medio, tiene alcance a todas las variables y no se le escapa nada. Esto no quiere decir que sea omnisciente.
- **Parcialmente observable:** Del otro lado hay sistemas que se entienden desde el principio que el conocimiento total de las variables que conforman el estado actual es imposible y que se debe manejar la indeterminación de alguna manera.

Determinista vs. Estocástico

- **Determinista:** El sistema produce la misma salida ante el mismo ingreso de estímulos.
- **Estocástico:** Ante la misma entrada, el resultado puede ser diferente y está sujeto a probabilidades. Cualquier cosa asociada a la estadística.

Episódico vs. Secuencial:

- **Episódico:** Cada "percepción y acción" es un evento **independiente**. Las acciones pasadas no afectan directamente el futuro. Piensa en clasificar emails: cada email es un episodio separado.
- **Secuencial:** Las acciones que el agente realiza en el presente influirán en las percepciones futuras. Un juego de ajedrez es secuencial: cada movimiento afecta el estado del tablero para los movimientos siguientes.

Estático vs. Dinámico:

- **Estático:** El entorno permanece inalterado mientras el agente "piensa" y decide qué hacer.
- **Dinámico:** El entorno puede cambiar mientras el agente procesa información o incluso mientras ejecuta una acción. Un coche autónomo opera en un entorno dinámico.

Discreto vs. Continuo:

- **Discreto:** Hay un número finito y bien definido de estados posibles, percepciones y acciones. Un juego de ta-te-ti tiene un número discreto de estados del tablero.
- **Continuo:** Los estados, percepciones o acciones pueden tomar un número infinito de valores. Conducir un coche implica percepciones continuas (velocidad, ángulo de giro).

Agente Individual vs. Multiagente:

- **Individual:** Solo un agente opera en el entorno.
- **Multiagente:** Hay varios agentes interactuando. Dentro de multiagente, puede ser:
 - **Competitivo:** Los agentes tienen objetivos opuestos y compiten (ejemplo: un juego de ajedrez con dos jugadores IA).
 - **Cooperativo:** Los agentes trabajan juntos para lograr un objetivo común (ejemplo: robots colaborando en una fábrica).

En entornos multiagente, puede surgir la **comunicación** entre los agentes.

Conocido vs. Desconocido:

- **Conocido:** El agente tiene una comprensión completa de las reglas y efectos de sus acciones en el entorno.
- **Desconocido:** El agente no tiene toda la información sobre el entorno y debe aprenderla a través de la experiencia.

TIPOS DE AGENTES

Agentes Reactivos: Son los más simples. Se basan en "reglas (conexiones si-entonces)". Es decir, perciben algo y reaccionan directamente sin mucha "reflexión" profunda.

- **Simple:** Solo consideran las percepciones actuales para tomar una decisión, ignoran el resto de las percepciones históricas. El agente funcionará solo si puede tomar la decisión correcta sobre la base de la percepción actual, lo cual es posible sólo si el entorno es totalmente observable.
- **Basados en modelos:** Mantienen un "estado interno" para almacenar información sobre partes del mundo que no pueden percibir directamente. Esto les permite tener una "**memoria**" del entorno. Es la forma más efectiva que tienen los agentes de manejar la visibilidad parcial. El agente debe mantener algún tipo de estado interno que dependa de la historia percibida, que permita reflejar por lo menos alguno de los aspectos NO observables. Se necesita alguna información sobre cómo evoluciona el mundo independientemente del agente y segundo se necesita más información sobre cómo afectan al mundo las acciones del agente.

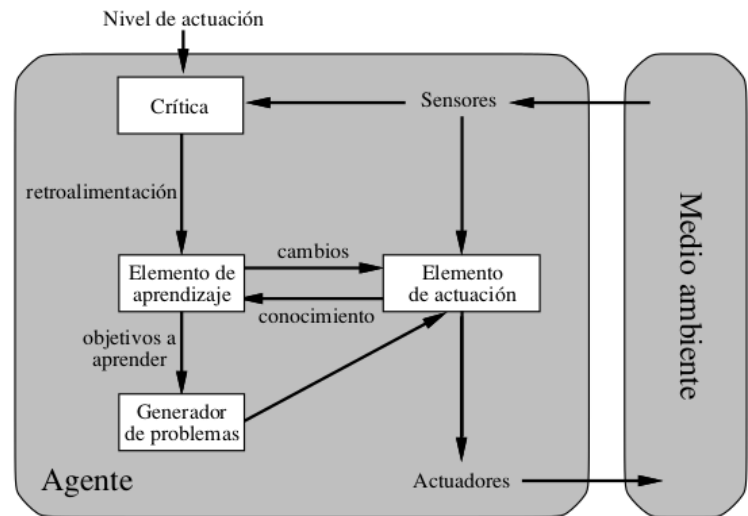
Agentes Basados en Objetivos: Actúan para alcanzar un objetivo específico (por ejemplo, llegar a un destino). El estado actual del mundo no es siempre suficiente para decidir que hacer, es decir, que además de la descripción del estado actual, el agente necesita información sobre su meta, que describa situaciones que son deseables.

Agentes Basados en Utilidad: Actúan para maximizar una función de "utilidad" o bienestar, buscando el mejor resultado posible en términos de valor o beneficio. Estos agentes utilizan una función de utilidad que proyecta del estado en un número real que representa el nivel de felicidad. La definición completa de una función de utilidad permite tomar decisiones racionales en casos en los que las metas son inadecuadas:

1. Cuando hay objetivos conflictivos y solo se puede alcanzar algunos de ellos, la función de utilidad determina el equilibrio adecuado.
2. Cuando haya varios objetivos por los que se pueda guiar al agente, y ninguno de ellos se pueda alcanzar con certeza, la utilidad proporciona un mecanismo para ponderar la probabilidad de éxito en función de la importancia de los objetivos.

Agentes que Aprenden: Son más complejos y pueden mejorar su rendimiento con la experiencia. Sus componentes principales son:

- **Elemento de actuación:** Es el responsable de seleccionar y ejecutar las acciones que lo guiarán hacia experiencias nuevas e informativas.
- **Elemento de aprendizaje:** Realiza mejoras en el agente a partir de la experiencia. Se realimenta de las críticas sobre la actuación del agente y determina cómo se debe modificar el elemento de actuación para proporcionar mejores resultados en el futuro.
- **Crítica:** Evalúa el rendimiento del agente y envía retroalimentación al elemento de aprendizaje.
- **Generador de problemas:** Sugiere acciones exploratorias para que el agente aprenda y descubra cómo mejorar su rendimiento en el futuro.



RECONOCIMIENTO DE PATRONES o MACHINE LEARNING – UNIDAD 1

La esencia de muchos problemas en IA (especialmente en Machine Learning) es encontrar una **función matemática o algorítmica** que actúe como un "mapeo" o "puente". Esta función toma ciertos datos que ya conocemos (llamados **variables de entrada** o *features*) y los transforma para producir un resultado (la **salida** o *target*).

Esta función puede ser un componente vital dentro de la arquitectura de un **agente inteligente**. Por ejemplo, un agente de navegación podría usar una función que reciba datos de sensores (entrada) y genere comandos de movimiento (salida). El "elemento de aprendizaje" del agente se encargaría de optimizar esta función para que el agente actúe de manera más "racional" o eficiente.

Se puede aplicar en un agente inteligente o como un fin en sí mismo, en esencia se trata de la estimación mediante una función que relaciona variables de entrada con la salida.

La función de estimación se construye con propósitos específicos, la **predicción**, comprender la relación entre las variables o en la mayoría de los casos, la **clasificación** en sí.

Predicción (o Regresión):

- El objetivo es estimar un **valor numérico continuo**. Esto significa que la salida puede ser cualquier número dentro de un rango.

Comprender la Relación entre las Variables o la Clasificación en Sí:

- **Comprender la relación:** A veces, el valor de la salida es menos importante que entender *cómo* las variables de entrada influyen en ella. Esto es clave en la investigación científica o en análisis de datos donde se busca conocimiento sobre un fenómeno.

- **Clasificación:** Este es un objetivo muy frecuente. En lugar de un valor continuo, la salida es una **categoría o una etiqueta discreta**. El modelo asigna la entrada a una de un conjunto predefinido de clases.

El reconocimiento de patrones es una función básica de los organismos vivos, un **patrón** es una descripción de un objetivo definida con una relación ente sus características. Es decir, lo que hace que algo sea un patrón son sus propiedades distintivas y cómo se organizan.

La naturaleza del "objetivo" que se busca reconocer influye en la forma en que se aborda el problema. Entre los tipos de objetos que existen tenemos:

- **Objetos concretos:** Reconocimiento perceptual. Implica procesar datos brutos de sensores para identificar estas entidades tangibles.
- **Objetos abstractos:** Reconocimiento conceptual. Implica interpretar y comprender el significado, la intención o la categoría de información más compleja.

APRENDIZAJE AUTOMÁTICO – UNIDAD 2

Cómo los sistemas pueden adquirir "inteligencia" a través del aprendizaje, en contraste con los agentes inteligentes cuya inteligencia es preprogramada por un diseñador. El aprendizaje automático se vuelve crucial cuando las reglas que rigen un entorno son desconocidas.

El núcleo del aprendizaje automático

El reconocimiento de patrones es una aplicación fundamental del aprendizaje automático. En esencia, busca establecer una función 'f' que vincule las variables de entrada, denominadas **características** con la variable de salida.

Los objetivos principales de esta estimación son la predicción, que a menudo se traduce en clasificación y la compresión de las relaciones entre las variables.

Dentro de este contexto, un "patrón" se define como la descripción de un objeto a través de las relaciones entre sus características. Los objetos a reconocer pueden ser tanto **concretos** (reconocimiento perceptual) como **abstractos** (reconocimiento conceptual).

Una de las herramientas que se utiliza para esto es Scikit-learn que es de Python3 y se usa para el aprendizaje automático (sin llegar a ser deep learning).

Para poder lograr el aprendizaje automático tenemos que primero **sensar**, ósea tomar de algún lado que sea confiable datos para obtener una representación confiable de los mismos. Esta etapa se refiere a descubrir diferentes cualidades de un objeto de estudio para conseguir su presentación entre rangos certeros.

Los datos sensados se representan mediante un vector $x = (x_1, x_2, \dots, x_n)$ donde x_i es el valor del atributo i .

Tipos de variables:

- **CUANTITATIVAS:** Son la medición mediante números en la cual se pueden usar enteros (discreto) o float (continuos) representando la intensidad del valor.

- **CATEGÓRICAS:** Conjuntos más limitados, más difíciles de filtrar muchas veces. Pueden ser ordinales, que pertenecen a una categorización por escala o nominales que son categorías sin jerarquía en donde es más cómodo asignarles a cada valor registrado un valor numérico.

ETAPAS O PASOS EN LA RESOLUCIÓN DE PROBLEMAS DE APRENDIZAJE AUTOMÁTICO

1. **Sensado (Obtención de Datos):** Esta es la fase inicial donde se recolectan los alores de los atributos relevantes del objeto a reconocer.
2. **Representación de los Datos:** Los datos obtenidos se organizan generalmente en un vector $x = (x_1, x_2, x_3, \dots, x_n)$ donde cada x_i es el valor de un atributo. Una vez que tenemos todos los datos es necesario que hagamos un análisis exploratorio, para poder entender que valores estamos recuperando y si hay anomalías.
 - a. **Variables Cuantitativas:** Se expresan numéricamente.
 - b. **Variables Categóricas:** Se les aplica la codificación one-hot, esta técnica expande una variable categórica original en m atributos binarios, donde m es la cantidad de valores posibles de la variable original.
3. **Extracción de Características:** Implica la creación, eliminación o reducción de la dimensión de las características y otras transformaciones. El escalado es una transformación útil para asegurar que todas las variables están en el mismo rango, como la normalización entre 0 y 1. Tecnicas como PCA (Análisis de Componentes Principales), Isomap y t-SNE son ejemplos de reducción de dimensionalidad que permiten visualizar y trabajar con datos complejos en un espacio de menor dimensión.

Estos tres puntos entran en lo que sería la preparación de los datos. Tras la obtención y extracción de características de los datos, las tareas principales son:

- **Clasificación:** Asignar un vector de características a una clase
 - **Regresión:** Predecir un valor continuo.
4. **Selección del modelo:** Se debe seleccionar un modelo de aprendizaje automático adecuado para el problema en cuestión. Esta selección dependerá en gran medida del tipo de problema que se este tratando de resolver y de las características de los datos. Una vez identificados los tipos de modelos mas adecuados para el problema, es necesario seleccionar el modelo más adecuado para la tarea en cuestión, evaluar el rendimiento de varios modelos en un conjunto de datos de prueba utilizando diferentes métricas, como la precisión, recall, F1-score y la curva ROC.
 5. **Entrenamiento del modelo:** Una vez que se ha seleccionado el modelo más adecuado para el problema, es hora de entrenarlo con el conjunto de datos de entrenamiento que hemos preparado. Antes de empezar a entrenar el modelo, vamos a dividir nuestro conjunto de datos en dos partes, el **conjunto de datos de entrenamiento** y el **conjunto de datos de prueba**. La idea es utilizar uno para ajustar los parámetros del modelo y el otro para evaluar su rendimiento en datos no vistos antes.

Existen dos tipos de entrenamiento (principales):

- a. **Entrenamiento supervisado:** Los parámetros se ajustan de forma tal que la salida del clasificador se aproxime a una salida esperada conocida. Es decir, se le proporcionan al algoritmo pares de entrada y salida correctas, y el algoritmo aprende a mapear las entradas a las salidas correctas.
- b. **Entrenamiento NO supervisado:** Los parámetros se ajustan para encontrar relaciones o agrupaciones entre los valores de las características, sin que se les proporcione una salida esperada conocida. El algoritmo busca patrones y estructuras ocultas en los datos por sí mismo.

6. **Clasificación:** En esta etapa se determina a qué clase pertenece cada vector de entrada y se generan límites de decisión entre las regiones que definen las clases. Los problemas pueden ser **linealmente separables** o **no linealmente separables**, lo que influye en la elección del clasificador.

CLASIFICADOR LINEAL – UNIDAD 1

El clasificador lineal pertenece al grupo de métodos de aprendizaje supervisado. Requiere un conjunto de vectores previamente clasificados (vectores de entrenamiento) para su ajuste.

El resultado de las etapas de sensado y extracción de características es un conjunto de vectores de características. Cada vector tiene la forma:

$$\mathbf{x} = [x_1, x_2, \dots, x_n]$$

Clasificación y Regresión

Un sistema de clasificación predice una categoría, mientras que un sistema de regresión predice un número.

La principal diferencia entre los algoritmos de regresión y clasificación es que los algoritmos de **regresión** se usan para predecir valores continuos como precio, salarios, etc. y los algoritmos de **clasificación** se usan para clasificar valores discretos como verdadero o falso, spam o no spam, entre otros.

- **Clasificación:** Asignar cada vector de características a una clase
- **Regresión:** Predecir un valor para cada vector de características.

Los métodos de aprendizaje automático que vemos en la materia son de clasificación, pero con pequeñas modificaciones pueden ser utilizados para regresión.

La **función de decisión lineal $d(\mathbf{x})$** se define como:

$$d(\mathbf{x}) = w_1x_1 + w_2x_2 + \dots + w_nx_n + w_{n+1}$$

Donde $\mathbf{x}$ es el vector de entrada con n características, y $\mathbf{w}$ es el vector de coeficientes o pesos. Si se utiliza un vector de entradas "aumentado" (añadiendo un 1 al final del vector $\mathbf{x}$), la función de decisión puede expresarse como un producto matricial **$d(\mathbf{x}) = \mathbf{xw}$** .

Para el entrenamiento de un clasificador lineal con dos clases (c_1 y c_2), se busca que $d(x) > 0$ para la clase c_1 y $d(x) < 0$ para la clase c_2 . Esto se logra **minimizando el error cuadrático medio**, y la expresión para calcular los **coeficientes óptimos** es $w = (X^T X)^{-1} X^T y$, donde X es la matriz de entradas y y es el vector de salidas esperadas.

Ejercicio 1:

Dados los siguientes vectores de entrada (00) (01) (10) (11) determinar la función de decisión lineal que realiza la correcta clasificación de las mismas según el siguiente vector de salidas $y = [1 \ 1 \ -1 \ -1]$

Determinar la función de decisión lineal.

Hay que encontrar:

$$d(x) = w_1 x_1 + w_2 x_2 + w_3$$

Para eso debemos encontrar el vector w , cómo? $w = (X^T X)^{-1} X^T y$

Creamos una matriz de entrada X con los vectores de entrada y lo aumentamos (agregamos 1)

001
011
101
111

Hacemos la transpuesta de la matriz:

0011
0101
1111

Multiplicamos ambas matrices:

0011
0101
1111
x
001
011
101
111
=
212
122
224

Se calcula la inversa de este resultado (suelen darla en los parciales.

Multiplicamos la inversa por la transpuesta.

Multiplicamos por los valores de salida.

Obtenemos los coeficientes de entrada.

CLASIFICACIÓN MULTICLASE: Se refiere a la tarea de asignar un elemento a una de varias clases posibles. Mientras que un clasificador binario distingue entre dos clases, un clasificador multiclase es capaz de diferenciar entre más de dos categorías. Para abordar este tipo de problemas con mas de dos clases, se suele utilizar estrategias que descomponen el problema multiclase en una serie de problemas de clasificación binaria más simples.

Contamos con dos estrategias para abordar la planificación multiclase:

1. Uno contra todos (One-vs-Rest o Piecewise):

- En este enfoque, cada clase tiene su propia función de decisión.
- Cada una de estas funciones se entrena de forma independiente para separar los vectores de una clase específica del resto de todas las demás clases.
- Cuando se presenta un nuevo vector al clasificador, la salida final se determina por la **función de decisión que produce el valor más alto para ese vector**. Esto significa que **el clasificador elige la clase cuya función de decisión le da la mayor "confianza"**.

2. Clasificación por pares (Pairwise o One-vs-One):

- Aquí, se crea **una función de decisión para cada posible par de clases**.
- El número de funciones de decisión necesarias es $N(N-1)/2$, donde **N es el número total de clases**. Por ejemplo, para 3 clases, se necesitarían $3(3-1)/2=3$ funciones de decisión.
- Cada función se entrena específicamente para separar los vectores pertenecientes a las dos clases de su par correspondiente.
- Cuando se clasifica un nuevo vector, se pasa por todas las funciones de decisión por pares. Cada función "vota" por una de las dos clases que está diseñando para distinguir. La salida final del clasificador es la clase que recibe la mayor cantidad de votos (la salida con la frecuencia más alta) de todas las funciones de decisión.

Ambas estrategias son métodos para extender clasificadores binarios (diseñados para dos clases) a problemas con múltiples clases.

FUNCION DE DECISIÓN GENERALIZADA – UNIDAD 1

La función de decisión lineal tradicional ($d(x)=w_1x_1+\dots+w_nx_n+w_{n+1}$) solo es efectiva para problemas donde las clases son **linealmente separables**.

Para abordar problemas más complejos, se introduce la **función de decisión generalizada**:

$$d(x) = w_1f_1(x) + w_2f_2(x) + \dots + w_mf_m(x) + w_{m+1}$$

Aquí, $f_i(x)$ son funciones reales simples del vector de entrada x . Esta generalización permite una flexibilidad infinita en la forma de la frontera de decisión. La clave está en la elección de estas funciones $f_i(x)$, que pueden ser desde **polinomios** hasta **funciones no lineales complejas**, lo que da lugar a diferentes tipos de clasificadores.

CLASIFICADOR POLINOMIAL

Un caso particular de la función de decisión generalizada es el **clasificador polinomial**. Para este tipo de clasificador, las funciones $f_i(x)$ son **monomios (términos individuales) de un polinomio**.

Ejemplo: Para un espacio bidimensional, un polinomio de grado dos podría incluir términos como x_1^2 , x_2^2 , x_1x_2 , además de x_1 y x_2 .

Función de decisión:

$$d(x) = w_{11}x_1^2 + w_{22}x_2^2 + w_{12}x_1x_2 + w_1x_1 + w_2x_2 + w_3$$
$$d(x^*) = wx^*$$

Los clasificadores polinomiales permiten generar **fronteras de decisión no lineales**, como círculos, elipses, hipérbolas, parábolas y otras formas curvas, lo que les permite separar clases que no son linealmente separables. El entrenamiento de estos clasificadores se realiza de la misma manera que los clasificadores lineales, pero con un conjunto de características transformadas.

MAQUINAS DE VECTORES DE SOPORTE (SVM)

Las SVM son **clasificadores binarios** (originalmente diseñados para dos clases) que tienen como objetivo encontrar la **frontera de decisión óptima** que **maximice el margen entre las clases**. A diferencia de otros clasificadores, las SVM **buscan la separación más "robusta"**, es decir, la que deja el **mayor espacio posible entre los puntos de datos de diferentes clases**.

- **Vectores de Soporte:** Son los puntos de datos más cercanos a la frontera de decisión. Estos puntos son cruciales porque definen el margen y, por lo tanto, la frontera óptima.
- **Kernel Trick (Truco del Kernel):** Las SVM pueden manejar problemas no linealmente separables utilizando funciones de "kernel". Estas funciones proyectan los datos a un espacio de mayor dimensión donde las clases se vuelven linealmente separables. Luego, se encuentra una frontera lineal en ese espacio de mayor dimensión, que al ser proyectada de vuelta al espacio original, se convierte **en una frontera no lineal compleja**. Ejemplos de kernels incluyen el lineal, polinomial, gaussiano (RBF) y sigmoide.

Aprendizaje No Supervisado: Agrupamiento, Competitividad y Autoorganización

A diferencia del aprendizaje supervisado, donde el algoritmo aprende de datos etiquetados (con salidas esperadas conocidas), el **aprendizaje no supervisado** trabaja con datos sin etiquetas. Su objetivo principal es **encontrar patrones, estructuras o agrupaciones** inherentes dentro de los datos.

K-means: Un Algoritmo de Agrupamiento Fundamental

El algoritmo K-means es uno de los métodos de **agrupamiento no supervisado** más populares y se utiliza para dividir un conjunto de datos en un número predefinido de "clusters" o grupos. El objetivo de K-means es encontrar grupos en los datos de tal manera que los puntos de datos dentro del mismo clúster sean lo más similares posible, y los puntos de datos en diferentes clústeres sean lo más disímiles posible.

Funcionamiento del Algoritmo K-means (paso a paso):

1. **Inicialización de Centroides:** El algoritmo comienza seleccionando un número k de puntos de datos aleatorios del conjunto como los "centroides" iniciales de los clústeres. Un centroide es el **punto central de un clúster**.
2. **Asignación de Puntos a Clústeres (Paso de Agrupamiento):** Para cada punto de datos en el conjunto, se calcula la **distancia** (generalmente euclidiana) a cada uno de los centroides. El punto de datos se asigna al clúster cuyo centroide es el más cercano. Se proporciona una función `kmeans_paso` que realiza esta asignación.
3. **Actualización de Centroides (Paso de Actualización):** Una vez que todos los puntos de datos han sido asignados a un clúster, los centroides se recalculan. El nuevo centroide de cada clúster es el **promedio de todos los puntos de datos asignados a ese clúster**.
4. **Iteración:** Los pasos 2 y 3 se repiten de forma iterativa. En cada iteración, los puntos se reasignan a los centroides más cercanos y los centroides se recalculan, haciendo que los clústeres se **refinen gradualmente**. El proceso continúa hasta que los centroides no cambian significativamente de una iteración a otra, o se alcanza un número máximo de iteraciones.

El algoritmo K-means es una herramienta poderosa para **descubrir grupos naturales dentro de conjuntos de datos no etiquetados**, basándose en la proximidad de los puntos a los centroides de los clústeres.

REDES NEURONALES – UNIDAD 2

La idea de las redes neuronales es simular directamente el funcionamiento del cerebro en una red neuronal digital que desde la perspectiva de las aplicaciones prácticas del reconocimiento de patrones, el realismo biológico, impondrá restricciones innecesarias (porque somos lentos).

No solo debemos simular sino mejorar los modelos físicos para convertirlo en características y funciones no lineales de estas características.

Modelo matemático - PERCEPTRON

Es un método de aprendizaje supervisado que realiza una clasificación binaria en base a la transformación lineal. El mismo presenta una neurona biológica donde las dendritas son las entradas, el axón es la salida siendo la sinapsis donde los coeficientes de la función de decisión y el comportamiento (simplificado) se replican acumulando la intensidad de los impulsos recibidos.

Si superan cierto umbral, activan la neurona emitiendo una salida como:

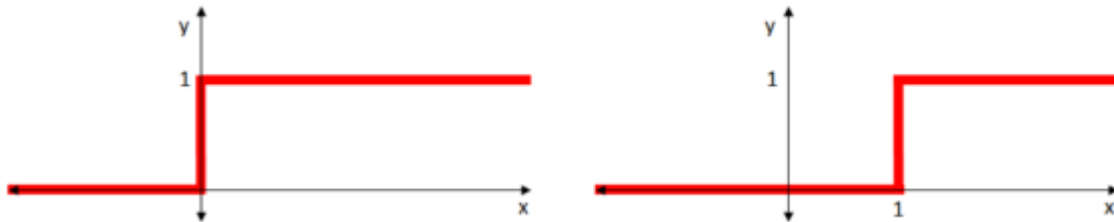
$$y = f \left(\sum_{i=1}^N w_i x_i - \theta \right)$$

x_i es el elemento i del vector de entradas $\mathbf{x} = [x_1, x_2, \dots, x_N]$

w_i es el elemento i del vector de pesos $\mathbf{w} = [w_1, w_2, \dots, w_N]$

θ es el umbral de activación.

f es la función umbral (también *threshold* o *hard-lim*). $f(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{si } z < 0 \end{cases}$



Pasos del método:

1. Definir el valor de alfa
2. Inicializar los pesos con valores aleatorios
3. Mientras error > 0 para algún vector de entradas, hacer:
 - a. Ejecutar ciclo completo de entrenamiento (epoch). Mientras existan vectores de entrenamiento hacer:

- i. Tomar un vector de entrada/salida del set de datos

$$y = f \left(\sum_{i=0}^N w_i x_i \right)$$

- ii. Calcular la salida

$$y' - y$$

- iii. Calcular el error

$$\Delta w_i = \alpha (y' - y) x_i$$

- iv. Calcular:

$$w_{i_{t+1}} = w_{i_t} + \Delta w_i$$

- v. Modificar los pesos haciendo:

Tal como está definido el método, a menos que se defina una cantidad máxima de ciclos, el entrenamiento termina cuando todos los vectores están bien clasificados.

El Perceptrón consiste en un solo nodo de procesamiento que acepta múltiples entradas y produce una única salida. El nodo realiza una serie de cálculos ponderando las entradas y sumándolas, y luego aplica una función de

activación para producir la salida. El algoritmo utiliza un conjunto de datos de entrenamiento para ajustar los pesos de las entradas, de manera que la salida se aproxime a la salida deseada.

Funcionamiento del Perceptrón Simple

Propagación hacia adelante: Las entradas se multiplican por sus respectivos pesos, se suman y se pasa la suma a través de la función de activación. La salida es entonces una predicción binaria (0,1).

Entrenamiento: Durante el entrenamiento el perceptrón simple ajusta sus pesos para que las predicciones se alineen lo mejor posible con las etiquetas de clases reales. Este ajuste se hace mediante un algoritmo de aprendizaje supervisado. Si la predicción es incorrecta, los pesos se actualizan en función de la diferencia entre la salida deseada $y_{deseado}$ y la salida real y_{real} :

$$w_i \leftarrow w_i + \eta (y_{deseado} - y_{real}) * x_i$$

donde η es la tasa de aprendizaje, un parámetro que controla la magnitud de ajuste.

Convergencia: El proceso de ajuste de los pesos continua iterativamente hasta que las predicciones del perceptrón para todas las muestras de entrenamiento sean correctas o hasta que se alcance un número máximo de iteraciones.

Usamos perceptrón para:

- **Clasificación Binaria:** Se utiliza principalmente para resolver problemas de clasificación binaria, como determinar si un correo es spam o no, o si una imagen contiene un objeto específico.
- **Problemas Linealmente Separables:** Es efectivo en problemas donde las dos clases se pueden separar por una línea en 2D o por un plano en 3D o un hiperplano en dimensiones superiores. Esto significa que el perceptrón simple solo puede resolver problemas que son linealmente separables.
- **Aprendizaje supervisado:** Es un modelo de aprendizaje supervisado, lo que significa que aprende a partir de un conjunto de datos etiquetados. Durante el entrenamiento, se ajustan los pesos del modelo para minimizar el error entre las predicciones del modelo y las etiquetas reales.
- **Fundamento para Redes Neuronales Más Complejas:** Si bien el perceptrón de una capa es bastante limitado, sirve como base para entender redes neuronales mas complejas. Es la base sobre la cual se construyen redes neuronales multicapa, conocidas como perceptrones multicapa o MLP, que pueden resolver problemas no lineales y realizar tareas mas complejas.
- **Limitaciones del Perceptrón Simple:** No puede resolver problemas NO linealmente separables. Como el problema XOR.
- **Capacidad Limitada:** Debido a su estructura simple, el perceptrón no puede aprender relaciones no lineales complejas entre las características de entrada y las salidas.

Funciones

FUNCIÓN ESCALÓN

Devuelve 1 si la entrada es positiva y 0 si es negativa

$$\begin{cases} 1 & \text{si } x \geq 0 \\ 0 & \text{si } x < 0 \end{cases}$$

FUNCIÓN SIGMOIDE

Convierte cualquier valor de entrada en un valor entre 0 y 1

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Utilizada en redes neuronales profundas y en la capa de salida para problemas de clasificación binaria.

FUNCIÓN TANGENTE HIPERBÓLICA (TanH)

FUNCIÓN ReLU (Rectified Linear Unit)

etc.

Si hay un error en la salida del perceptrón, lo que se tiene que modificar para corregir la salida son los pesos del perceptrón. La corrección se realiza mediante el proceso de entrenamiento que ajusta los pesos en función del error observado.

Proceso de corrección del error

Calcular el error: $e = y_{\text{deseado}} - y_{\text{real}}$

Actualizar los pesos: Los pesos w_i se ajustan utilizando una regla de aprendizaje como la regla delta o la regla de aprendizaje del perceptrón que generalmente es:

$$w_i \leftarrow w_i + \eta (y_{\text{deseado}} - y_{\text{real}}) * x_i$$

Donde w_i es el peso asociado a la entrada x_i , η la tasa de aprendizaje y x_i la entrada correspondiente.

Actualizar el sesgo (bias) si aplica:

Si el perceptrón tiene un término de sesgo, este también se ajusta de manera similar:

$$b \leftarrow b + \eta (y_{\text{deseado}} - y_{\text{real}})$$

Este proceso se repite para cada muestra de entrenamiento hasta que el perceptrón clasifique correctamente las entradas (o hasta que se cumpla un criterio de parada, como un número máximo de iteraciones o una reducción suficiente del error)

Modelo matemático - ADALINE

Este modelo matemático Adaline refiere a ADaptative LINear Element que es otra forma de armar redes neuronales con diferencias al perceptrón. Primero que su salida no es 1 o 0 en base a si es mayor a un umbral, sino que puede utilizar otras funciones de activación dependiendo el contexto. Esto le permite resolver situación de **regresión lineal**, en nuestra cátedra se establece que el Adaline es lineal y que por fuera del modelo usamos una **función umbral** para determinar la clase. Esto quiere decir que calcula igual que el perceptrón y la activación dependerá de una $d(x)$ que nosotros queramos.

Otra cosa que cambia con respecto en Adaline es que el aprendizaje de los mismos está basado en una regla delta del mínimo error cuadrático medio (Least Mean Squared o LMS error). Los pesos de la red se ajustan mediante una búsqueda guiada por el gradiente descendiente de la función de error.

$$O = \sum_{i=0}^N w_i x_i = Net.$$

Salida de la red (por output)

$$E = \frac{1}{2L} \sum_{k=1}^L (y_k - O_k)^2,$$

Error cuadrático medio

Donde “k” es el identificador de un vector de entrenamiento y “L” es la cantidad de vectores de entrenamiento. “E” es nuestro error cuadrático medio.

La adaptación de los pesos se lleva a cabo mediante una búsqueda sobre la superficie del error. Para encontrar el mínimo se debe modificar en cantidades proporcionales al gradiente decreciente de la función “E”.

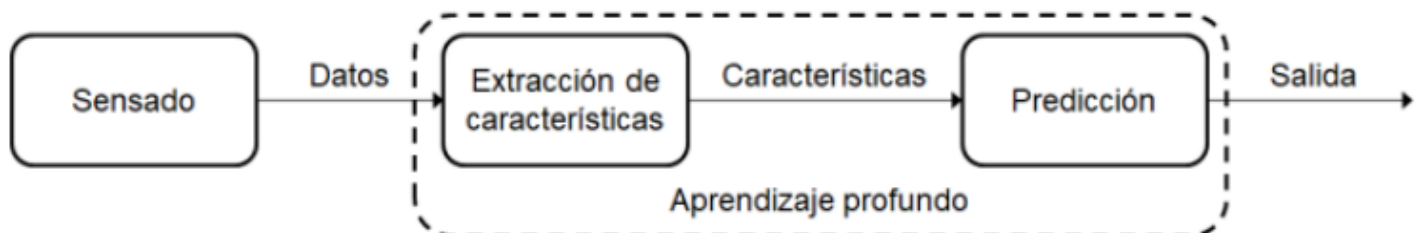
$$\begin{aligned}\Delta_{w_{ik}} &= -\alpha \delta x_i \\ &= -\alpha (-(y_k - O_k)) x_i \\ &= \alpha (y_k - O_k) x_i\end{aligned}$$

Finalmente, la expresión de la ley de aprendizaje de Adaline es igual a la del perceptrón:

En el caso de Adaline el error puede tomar **cualquier valor en R**. Para hacerte un breve resumen de esto, lo que te importa es el final, notarás que las funciones de error delta son iguales y es porque es así. Lo único que va a cambiar es que **en vez de tener corrección de errores de -1 o 1 tenemos todos los reales que existan entre la salida deseada y la esperada** (Cualquier valor real). Ahora el chiste que debemos recordar por si preguntan; es que **la diferencia al error nunca es cero** por ende no se estabiliza cuando el error de 0. Lo que se hace es establecer una **función de corte** que es nuestro umbral de error previamente definido. El otro concepto que te suelen tirar es que **al ser lineal no tiene sentido hacer multicapa ya que la conexión de muchas de estas se puede resumir en una sola transformación lineal a la salida**. Por lo que conectar neuronas es lo mismo que hacer uno solo con una función que transforme de manera equivalente todo.

APRENDIZAJE PROFUNDO – UNIDAD 2

El aprendizaje profundo (deep learning) es una evolución de las redes neuronales que venimos hablando. El término proviene de que ahora hacemos redes neuronales con mayor cantidad de capas (más profundas) donde este campo ha superado a los sistemas de aprendizaje automático con funcionalidades ad hoc (Un solo propósito). Esto quiere decir que estos nuevos sistemas son más complejos y son multipropósito para el reconocimiento de patrones. Su mayor virtud reside en aprender representaciones más complejas de características y ajustar los parámetros del clasificador al mismo tiempo que esto ocurre.



El neocognitron, creado en 1982 por Fukushima, es considerado por muchos autores el primer modelo de aprendizaje profundo. Es un modelo de extremo a extremo que reconoce caracteres escritos a mano que intenta imitar el procesamiento de señales en la corteza visual de los mamíferos. Esto lo hace por capas simples parecidas a las de **convolución y pooling** (que veremos a continuación) y lo que hace es reconocer jerárquicamente **patrones visuales simples en las primeras capas** y después **más complejos en las siguientes**. En 1989 con la invención del modelo de retropropagación se comienza una nueva era en las redes neuronales.

Convolución y Pooling

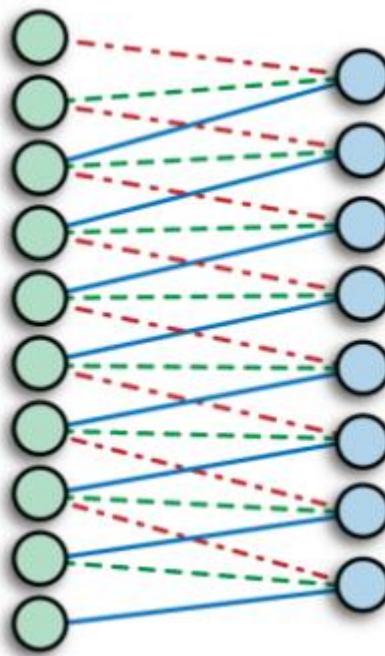
Las capas de convolución son básicamente una transformación lineal de sus entradas antes de una función de activación.

La salida de la neurona j se define como:

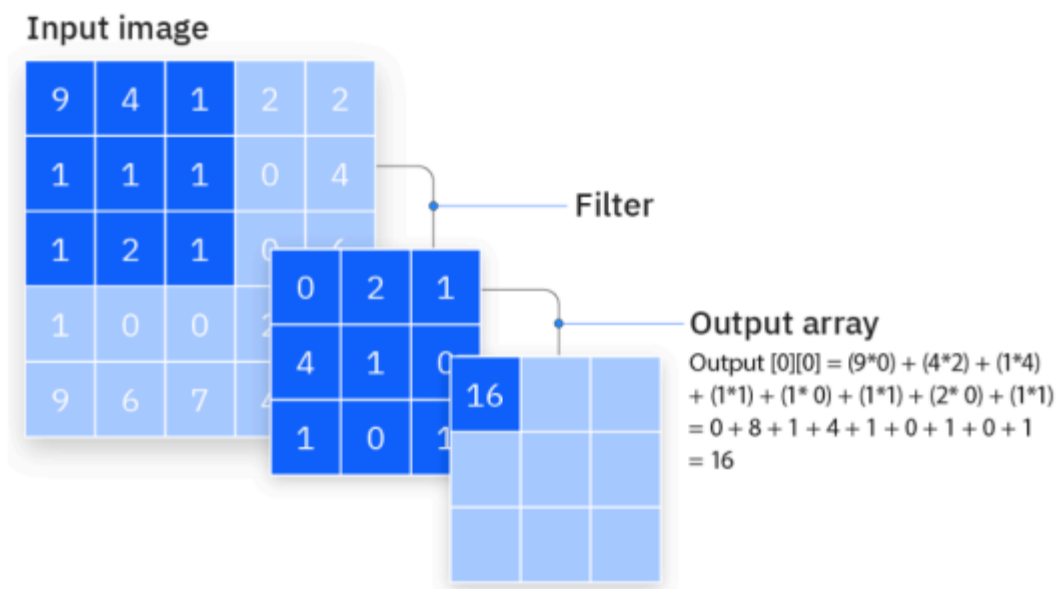
$$s_j = f\left(\sum_{i=0}^N w_{ij} x_i\right)$$

Donde “ $f(\dots)$ ” es la **función de activación**, “ x_i ” es la entrada “ i ”, “ w_{ij} ” es el peso sináptico correspondiente a la entrada i de la neurona j y w_{0j} es el término independiente o bias de la neurona j ($x_0 = 1$).

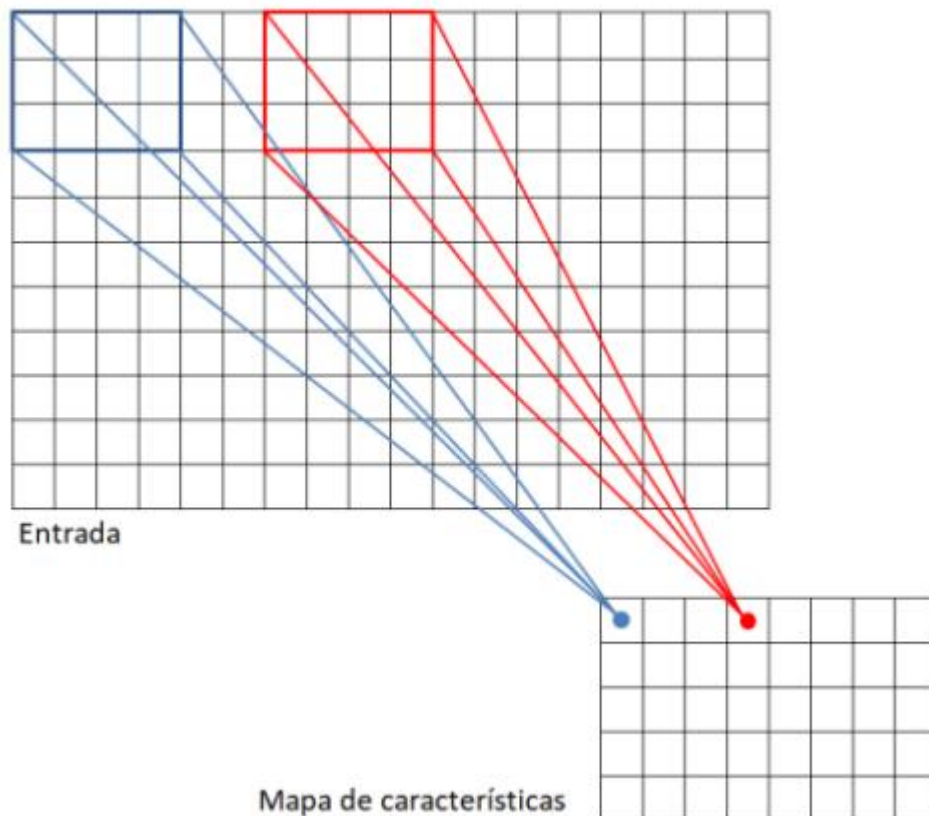
Estas neuronas se organizan como mapas de características (feature maps) dentro de las cuales cada unidad está conectada a regiones determinadas del mapa anterior mediante la utilización de un conjunto de pesos (Kernel o Filter bank). Todas las neuronas de un mapa (capa) comparten un mismo kernel y no todas las neuronas se conectan a la capa anterior como veíamos en redes neuronales.



Como vemos cada neurona tiene un “campo receptivo” acotado lo que permite manejar la información de una manera diferente a la que veníamos viendo. Lo que permite esto en términos cortos es que podamos detectar patrones en cualquier parte del arreglo (o imagen).



Otro concepto importante es el **pooling**, es una operación sobre una matriz para transformarlo en otra con algún tipo de importancia.



Para poner un ejemplo tenemos el MaxPooling que refiere que a una matriz le apliquemos cada cierto cantidad de bloques una operación MAX. Osea que hacemos lo mismo que la convolución, pasamos un “lente” y el resultado en una matriz generalmente más chica es el mayor número de ese conjunto.

Ahora mientras que la función de la capa de convolución es detectar conjunciones locales de la capa anterior el pooling es la fusión de varias características en un solo valor. En las redes de convolución (CNN) es común que se combinen ambas de manera intercalada. La retropropagación del gradiente del error a través de la red es simple por lo que permite entrenar los pesos de todos los kernels.

REGULARIZACIÓN (EVITAR SOBREAJUSTE) – UNIDAD 2

Sobreajuste implica que nuestro algoritmo se ajuste demasiado al set de entrenamiento y sea incapaz de obtener buenos resultados con datos reales. Si bien en el entrenamiento va a copiar los resultados con sus predicciones hace que nuestro algoritmo sea ineficiente con el set de datos de validación o prueba.

Notamos sobreajuste cuando tenemos pocos errores con el set de datos de entrenamiento y muchos errores con respecto al de prueba.

Las redes neuronales profundas son modelos complejos, con muchos parámetros para ajustar, lo que las hace propensas a que nuestro modelo escuche el “ruido”. Es entonces cuando definimos la **regularización** como la **modificación que hacemos en el aprendizaje de un algoritmo** para reducir los errores generalizados pero no de entrenamiento. Lo que intenta es no cambiar como calculamos los pesos por un error sino cambiar parámetros generales para mejorar la eficacia del algoritmo.

Existen cuatro estrategias para mejorar los resultados generales:

- Early Stop
- Penalización
- Dropout
- Aumentación de datos

EARLY STOP

Esta técnica es útil para entrenar modelos grandes con suficiente capacidad de representación para que podamos tener un **sobreajuste**. Se utiliza cuando vemos que nuestro **error fluctúa** o en su defecto el error final es mayor que el mínimo del modelo. Se guarda la configuración de los parámetros en aquellas iteraciones con menor cantidad de error. Una vez finalizado el entrenamiento se revisa **cuál fue dicha configuración** y se “**restaura**”. Se llama early stop por eso mismo, **iteramos hasta el final pero volvemos a cuando el modelo tenía menos error**.

PENALIZACIÓN

Otra estrategia es imponer restricciones adicionales al modelo de aprendizaje automático típicamente se agregan restricciones a los parámetros como términos extras de la función objetivo. Agregamos valores que sacan peso de las variables de decisión. Es decir, si consideramos que ciertos valores no deberían influir tanto en nuestro modelo y notamos que llevan a predicciones incorrectas, es entonces cuando los “**penalizamos**” o **regularizamos** para que cambie el ajuste del error.

- Regularización L1

$$\Omega(\mathbf{w}) = \beta \sum_{i=1}^N |w_i|$$

donde β es una constante que pondera la contribución relativa de la penalidad.

- Regularización L2

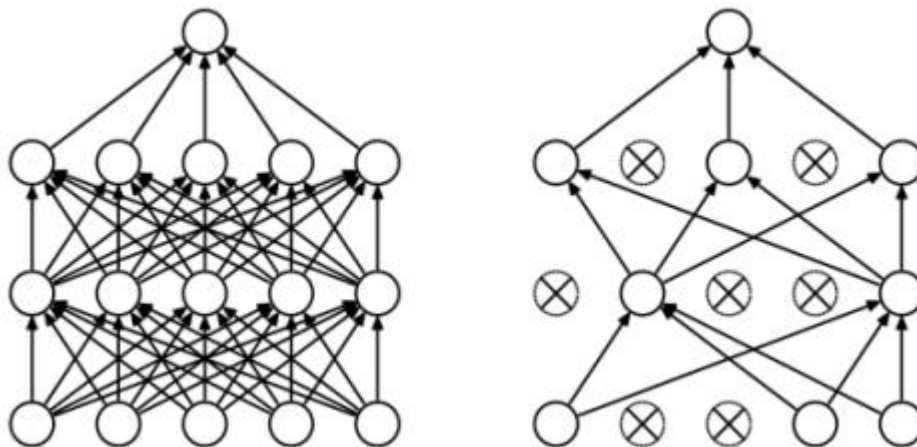
$$\Omega(\mathbf{w}) = \beta \sqrt{\sum_{i=1}^N |w_i|^2}$$

- Regularización L1 y L2

$$\Omega(\mathbf{w}) = \beta \sum_{i=1}^N |w_i| + \gamma \sqrt{\sum_{i=1}^N |w_i|^2}$$

DROPOUT

Es un concepto extraño, en cada ciclo con probabilidad definida se anulan de cada capa cierto número de neuronas. Básicamente deja inoperativas neuronas al azar, x número de neuronas y se ve si mejora el rendimiento. Cómo son modelos complejos esto puede resultar en que mejore el rendimiento general ya que la neurona estaba siendo parte del sobreajuste. Acá en la figura vemos como se anulan en diferentes capas las neuronas:



AUMENTACIÓN DE DATOS

Tenemos que aumentar la cantidad de datos para obtener una mejor generalización. También se puede mejorar el set de datos buscando randomizar más el dataset de entrenamiento para encontrar más valores dispersos. Algo que dice la guía que se puede hacer es modificar los datos actuales levemente para obtener nuevos resultados.

Esto no solo es útil para que tengamos más datos en magnitud, sino que además entremos a la IA a reconocer patrones con pequeñas alteraciones y hacerlo más perspicaz en los reconocimientos.

MÉTRICAS - UNIDAD 2

Las métricas de evaluación en modelos de clasificación permiten **medir el desempeño** y la **capacidad del modelo** para **predecir correctamente las clases**. Es la forma en que tenemos de medir que tan acertado está siendo nuestro modelo.

Matriz de Confusión

La matriz de confusión es una tabla que se utiliza para describir el **rendimiento** de un modelo de clasificación. Presenta el número de **verdaderos positivos (True Positives)**, **verdaderos negativos (True Negatives)**, **falsos positivos (False Positives)**, y **falsos negativos (False Negatives)**. (Muestra la distribución de clasificaciones correctas e incorrectas en términos de TP, FP, TN y FN).

	Predicción: Positivo	Predicción: Negativo
Real: Positivo	TP	FN
Real: Negativo	FP	TN

Precisión (Precision)

La precisión de una clase define que tan confiable es un modelo en **responder que un punto pertenece a esa clase**. Es la proporción de **TP / (TP+FP)**. Mide cuan precisas son las predicciones positivas del modelo.

Exactitud (Accuracy)

La exactitud del modelo es el numero total de predicciones correctas dividido por el numero total de predicciones. Proporción de instancias correctamente clasificadas sobre el total de casos.

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Sensibilidad (Recall)

El recall representa cuan bien puede el modelo detectar a esa clase. Mide la capacidad del modelo para identificar correctamente las instancias positivas. Es la proporción de verdaderos positivos sobre el total de positivos reales **TP / (TP + FN)**

Especificidad (Specificity)

Mide la capacidad del modelo para identificar correctamente las instancias negativas. Proporción de verdaderos negativos sobre el total de negativos reales **TN / (TN + FP)**

Tasa de Falsos Positivos (FPR)

La tasa de falsos positivos es la proporción de falsos positivos sobre el total de negativos reales **FP / (FP + TN)**

F1 Score

Es la media armónica entre precisión y sensibilidad **(2 * (Precisión / Recall)) / (Precisión + Sensibilidad)**.

Combinar precisión y recall en una sola métrica es útil cuando las clases estan desbalanceadas.

Curva ROC

La curva ROC es una gráfica que muestra la relación entre la tasa de verdaderos positivos (TPR o Sensibilidad) y la tasa de falsos negativos (FPR). Permite visualizar el desempeño del modelo en varios umbrales de clasificación. Es una herramienta visual.

- **Línea diagonal:** Clasificador aleatorio, un clasificador que no tiene capacidad de discriminación, simplemente adivina.
- **Curva por encima de la diagonal:** Clasificador útil. Cuanto mas lejos este la curva de la línea diagonal y mas cerca del punto (0,1) mejor el clasificador. Si el modelo es perfecto, la curva pasaría por el punto (0,1) lo que indica una alta sensibilidad y baja tasa de falsos positivos.
- **Curva por debajo de la diagonal:** Clasificador malo. Rendimiento peor que adivinar al azar. Esto podría indicar que el modelo necesita mejoras o que las etiquetas están mal asignadas.

AUC (Área debajo de la Curva ROC)

Cuantifica el desempeño global del modelo. Mide la capacidad de discriminación del clasificador (un valor más alto es mejor). Un AUC de 1 indica un modelo perfecto, mientras que un AUC de 0.5 indica un modelo que no tiene mejor desempeño que un clasificador aleatorio.